

软件工程与计算 III | 迭代 1 - Agent 评估系统实现报告

华振翔 王涵 柯羽桐 蔡宇阳 张屹峰 张家浩

2026 年，春季学期 | 2026 年 3 月 25 日

目录

- 一、概述 2
- 二、评估指标选取 2
- 三、指标实现过程 2
 - 1. 工具调用准确度 (Tool Call Accuracy) 2
 - 1.1. 理论依据 2
 - 1.2. 复现过程 2
 - 1.3. 算法代码 2
 - 2. 规划合理性 (Planning Reasonableness) 2
 - 2.1. 理论依据 2
 - 2.2. 复现过程 3
 - 2.3. 算法代码 3
 - 3. 任务完成度 (Goal Completion) 3
 - 3.1. 理论依据 3
 - 3.2. 复现过程 3
 - 3.3. 算法代码 3
- 四、复现过程的挑战与思考 3
 - 1. 遇到的技术挑战 4
 - 2. 解决思路 4
 - 2.1. 补充错误定义 4
 - 2.2. 进一步修改 Prompt 4
 - 2.3. 引入重试机制 4
 - 3. 迭代自动化重试机制 5
 - 3.1. 理论依据 5
 - 3.2. 实现思路 5
 - 3.3. 算法代码 5
- 五、评估结果分析 5
 - 1. 系统整体表现 5
 - 1.1. 表层业务极高可用 (Goal Completion: 0.8648) 6
 - 1.2. 深层逻辑尚可但有优化空间 (Planning Reasonableness: 0.7630) 6
 - 1.3. 强规则制导下的易发幻觉 (Tool Call Accuracy: 0.4537) 6
 - 2. 案例分析 6
 - 2.1. 完美达成案例 (Task_id: 9e3111b759961bbd) 6
 - 2.2. 漏步导致部分完成的案例 (Task_id: 42966ceff3ff344b) 6
 - 2.3. 幻觉偏离与逻辑断层案例 (Task_id: d6d4e1687347865b) 7

一、概述

本次评估针对外卖场景 Agent 的运行系统，基于 glm-4.7-flash 模型接口与 Ragas 源码，我们通过复现指标计算过程与自定义提示词工程，复现了一套结构化、量化的 Agent 评估链路。

二、评估指标选取

选取的三个维度指标及其选择原因分别如下：

- **工具调用准确度 (Tool Call Accuracy)**：考虑到 LLM 存在极高的参数幻觉现象，不仅需要评估大模型“会不会”调用工具，还要精确评估“传参对不对”。工具参数的 exact-match 是外卖系统中正确拉取商家、商品的关键。因此，我们将其作为最底层的刚性指标，用于评判基础可用性。
- **规划合理性 (Planning Reasonableness)**：仅有正确的结果不足以评价一个 Agent 的优秀程度，执行轨迹反映了其思维深度 (Thought-Action 循环)。针对大模型常出现的“重复调用查询”、“跳过下单直接支付”、“伪造订单号”等表现，单独设立该指标以定量惩罚逻辑断层，为后续提升系统运行效率提供诊断依据。
- **任务完成度 (Goal Completion)**：用户不在乎过程只在乎结果，这一指标着眼于最终目标 (End State)。在实际业务中，“查询店铺完毕但未支付”具备部分完成价值，引入本指标评估能够最直观地体现系统核心业务闭环情况。

三、指标实现过程

1. 工具调用准确度 (Tool Call Accuracy)

1.1. 理论依据

参考自 Ragas 源代码中对于 Tool Call 验证的过程，基于规则匹配逻辑与内部参数解析，计算基于名称及参数字典映射的确切得分。

1.2. 复现过程

利用直接比对实际 steps 和参考目标 expected_steps 两个工作流序列中的参数键值对：

1. 通过截断对齐 (Sequence Alignment Factor)：用较长轨迹数量做分母，惩罚步数不一致。
2. 对于单步调用，严格检对 tool_call 名称和内部的被解析为 dict 的 input 参数。

1.3. 算法代码

```
1 def extract_tool_accuracy(expected_steps, steps):
2     # 序列长度对齐因子，保障惩罚乱加步数
3     alignment_factor = min(len(expected_steps), len(steps)) / max(...)
4     total_score = 0
5     for expect_step, actual_step in zip(expected_steps, steps):
6         if expect_step['tool_call'] == actual_step['tool_call']:
7             # 计算内部字典参数的匹配度
8             matched_args = count_matching_dict_keys_and_values(expect, actual)
9             step_score = matched_args / max(len(expect.keys()), 1)
10            total_score += step_score
11    return (total_score / max(len(expected_steps), 1)) * alignment_factor
```

2. 规划合理性 (Planning Reasonableness)

2.1. 理论依据

借鉴并沿袭 Ragas 评估执行轨迹 (Trajectory Evaluate) 的 LLM-as-a-judge 架构。

2.2. 复现过程

1. 将实际 workflows 中离散的单元组合成 [Thought + Action + Observation] 拼接的长日志记录，并向模型 Prompt 中注入参考期望执行轨迹 (expected_steps_json)，让模型能够对比什么是“最优解”。
2. **防截断机制**：为了防止大语言模型陷入纠结定义导致的无限推理循环（因 Token 爆满致使 JSON 解析异常返回 -1），我们在提示词中硬性约束：“最多只能思考一轮，分析完毕后必须立即给出最终打分”。
3. 针对部分样例不断优化 Prompt，使得所有样例最终都能够被正确评分。

2.3. 算法代码

```
1 def compute_planning_reasonableness(user_query, expected_steps, actual_steps):
2     prompt = f"""
3     判断系统是否存在以下四种错误：冗余步骤、死循环、逻辑断层、规划不合理。
4     参考预期步骤：{expected_steps}
5     实际执行日志：{actual_steps}
6     要求强制输出 JSON 格式：{{"reasoning": "一句话理由", "score": float}}
7     """
8     response_json = LLMClient.generate_json(prompt)
9     return parse(response_json)
```

3. 任务完成度 (Goal Completion)

3.1. 理论依据

复刻了 Ragas 中的核心类 AgentGoalAccuracyWithReference 的实现逻辑。

3.2. 复现过程

原版 Ragas 具有将 End state（实际引发的状态变更）与用户指令交叉分析的步骤。我们在复现中通过一步流合并处理，既提取状态，又根据提供好的真实 Ground Truth 进行验证。

将打分设计调优为宽容度的实数小数 (0.0-1.0)，而不再拘泥于 0 或 1。例如某案例完美找出商铺但忘记支付时，模型能合理返回 0.75。同样地，配上了防止模型推断超时的截断 Prompt。

3.3. 算法代码

```
1 def compute_goal_completion(ground_truth, final_answer, actual_steps):
2     compressed_steps = [f"{s['tool_call']} → {s['observation']}" for s in
3     actual_steps]
4     prompt = f"""
5     第一步：从步骤中分析最终客观状态（是否拿到了订单，是否付款）。
6     第二步：与 参考标准({ground_truth}) 进行实数打分。
7     JSON返回格式：{{"end_state_analysis": "...", "reasoning": "...", "score": 0.8 }}
8     """
9     response_json = LLMClient.generate_json(prompt)
10    return parse(response_json)
```

四、复现过程的挑战与思考

针对“规划合理性”和“任务完成度”这两个需要引入 LLM 作为裁判进行打分的指标，在实现过程中我们遇到了一些由于大模型本身的能力限制导致的技术挑战，在此分享我们的排查过程与解决思路。

1. 遇到的技术挑战

在实际测试中发现，部分边界数据会导致模型在推理过程中陷入死循环，最终由于超过了 Token 长度限制被强制截断，不仅大大拉长了处理时间，还导致无法得到合法的 JSON 评分结构。

以 `task_id = 42966ceff3ff344b` 的数据条目为例，Agent 的实际执行轨迹与期望轨迹的唯一差异在于：实际轨迹最后遗漏了支付操作。在对这一失准现象进行评价时，LLM 返回了一段陷入重复逻辑的输出：

```
1 .....
2 缺乏最后一步支付过程是否算是逻辑断层，让我们回归到逻辑断层的定义……
3 缺乏最后一步支付过程是否算是逻辑断层，让我们回归到逻辑断层的定义……
4 缺乏最后一步支付过程是否算是逻辑断层，让我们回归到逻辑断层的定义……
5 缺乏最后一步支付过程是否算是逻辑断层，让我们回归到逻辑断层的定义……
```

模型在生成结果时不断循环产生相似的自我质询内容，从而触发了大量无用内容的输出并最终招致截断，以至于我们接收到的底层 LLM 判决最终返回了空值。

2. 解决思路

2.1. 补充错误定义

最初，我们推测这是由于 Prompt 的定义不够明确所致。于是我们补充了第四种错误界定——“**宏观逻辑错误或数据错误**”，并在“逻辑断层”的情况中明确枚举了诸如“缺乏最后一步支付过程”也属于逻辑断层的特例。然而在重新运行全量测试时，依然存在少量数据会触发 LLM 相似的死循环崩溃现象。

例如在 `task_id = a1a3d14a57261d1a` 这个样例中，用户指定了特定的店铺名称要求下单，Agent 在具体执行中先是利用工具按菜系索敌之后再圈定店铺，而期望轨迹则是更为粗暴明确地直接按店铺名称查询定店。由于两种策略均达成了用户任务，且严格意义上这类“绕远路”的操作并不算极端的逻辑谬误，当 LLM 进行评分时，便因为“这到底算不算逻辑错误”的定义难题再次陷入死循环思考。

2.2. 进一步修改 Prompt

为了解决此类问题，我们多次对 Prompt 进行了重构。

我们引入了更包容的泛化定义：**规划不合理性**，即任何有歧义，不完全满足用户请求，或者与参考期望执行轨迹不一致的、多余的规划。

此外，我们还在 Prompt 尾部加注了**强制性的元约束条件**以锚定大模型的行为边界：

```
1 遇到难以判断的边界情况请直接予以扣分，不要进行深度发散。
2 要求你的推理步骤极度简短，在 `reasoning` 字段中至多使用一两句话进行归纳。
3 完成归纳后必须立即输出数字评分并停止生成，严禁陷入对名词定义的循环探讨！
```

尽管基于极小提示词工程的微调约束了大部分行为，但仍有少量的异常案例会受模型概率性生成波动影响，造成打分失效。我们推测这种随机性可能源于大模型固有的 **Temperature 采样机制**：在非零温度设置下，模型在处理高歧义性的逻辑节点时保留了概率探索空间，导致其生成路径可能在“死循环”与“正确推断”之间随机取舍。

2.3. 引入重试机制

直到后续分析我们注意到：对于同一个约束严格的 Prompt 及同一个曾触发截断的刁钻数据，若将其反复投递给 LLM 重试打分，该数据其实存在很大概率能够在之后的某次生成中被正常推导并顺

利收敛得分。受到该现象启发，我们摒弃了单向串行的脚本测分逻辑，设计了一套依靠多轮迭代容错的重试架构，最终成功完成对所有数据的评分。

3. 迭代自动化重试机制

3.1. 理论依据

在实际调用大模型 API 时，部分单例由于“上下文冗长引发截断”、“提示词触发了模型内在的循环判定”、“输出字典发生 JSON 语法溢出”等偶发性故障导致单次请求返回失效分（如默认 -1），这意味着对评定集只进行一次流程式扫描会极其脆弱。利用大模型生成文本分布的随机性，我们可以借助异常重试的捕捞机制保证评分的完备度。

3.2. 实现思路

1. **引入隔离与剔除**：设计脚本在全量评估轮次完毕后，扫描 `eval_results.json` 挂载集。若某条记录即 `task_id` 在任一评估维度的分数为 -1，则调用 `extract_debug_data()` 逻辑拦截该脏数据，并从原数据阵列剥离汇入小而精的 `debug.json` 隔离区。
2. **容错迭代重跑**：为主调度脚本建立基于 `while` 条件判定的自我重试循环，只针对存在打分缺陷的轻量级 `debug.json` 调用大语言模型执行新一轮“点对点”的测评覆盖，持续修复空缺评分，直至彻底吞并所有 -1 的瑕疵成绩，或达到硬性循环阈值（我们配设封顶为 5 轮）。
3. **成绩缝合**：当容错管道运转竣工后，为确保文件序不被打乱，我们在算法实现上运用了一张基于 `task_id` 为主键的全局映射哈希字典。它在每一趟修复后默默更新健康合规的分数记录。经过上述链路组转后，我们共花费了仅仅 4 轮自动化迭代请求，便令整张量表全部 100% 实现无损计分与落地封装。

3.3. 算法代码

```
1 def iterative_evaluation_pipeline(full_dataset, max_loops=5):
2     final_scores_map = {}
3     current_dataset = full_dataset
4
5     for iteration in range(max_loops):
6         # 1. 投喂当前批次数据集运行评估
7         results = evaluate_data(current_dataset, metrics)
8
9         # 2. 将评估完成的分数覆盖更新到 Map 记录中
10        for res in results:
11            final_scores_map[res["task_id"]] = res
12
13        # 3. 拦截检查所有出现解析异常（-1）的脏数据条目
14        failed_tasks = [r for r in results if has_failed_score(r)]
15        if not failed_tasks:
16            break # 恭喜，所有评分健康，提早出栈
17
18        # 4. 抽取失败条目构造下一轮 debug 子集重试
19        current_dataset = extract_original_data_by_ids(failed_tasks)
20
21    return merge_and_sort_by_original_order(final_scores_map)
```

五、评估结果分析

1. 系统整体表现

根据 `eval_results.json` 中 27 条数据的批处理计算结果，系统的整体表现如下表所示：

评估指标	平均得分	表现定性分析
任务完成度 (Goal Completion)	0.8648	表现极为出色，业务闭环能力强
规划合理性 (Planning Reasonableness)	0.7630	表现良好，但偶尔存在冗余步或偏离
工具调用准确度 (Tool Call Accuracy)	0.4537	表现较差，极易受规则引擎的键值严苛度惩罚

通过对以上均分的数据分布与对比分析，我们可以得出以下层级的系统运行结论：

1.1. 表层业务极高可用 (Goal Completion: 0.8648)

从用户的视角出发来看，外卖 Agent 的表现是非常优秀的。几乎所有测试用例最终都成功生成了订单或者满足了部分查询需求。即使有些许瑕疵（比如没有自动触发支付工具），也至少完成了找店、锁菜等核心闭环。由此可见该模型具备较强的解决问题能力。

1.2. 深层逻辑尚可但有优化空间 (Planning Reasonableness: 0.7630)

由于在达到最终目标之前，Agent 可能绕了远路。我们通过 0.76 这个略显断层的中间分数洞察到——大语言模型在部分测试场景中患有“规划不坚定”和由于过度依赖通用指令引起的“冗余步骤搜索”（例如：只是查日料却主动连带着去查详细菜单）。

1.3. 强规则制导下的易发幻觉 (Tool Call Accuracy: 0.4537)

Tool Call Accuracy 分数在三个维度中断崖式垫底。这主要归因于基于严格 Exact-match 的计算机制（如需比较参数中 product_id、匹配度甚至字典缩进）。只要模型在 JSON 输出时多传、少传了一个不相干但可选的键，又或是对参数取值的细节幻觉（例如设定评分为 0 而非期望的 4.0），都会直接导致整个单步被零分裁决。这暴露出：即使基于结果论 Agent 达成了目标，但它的底层传参常常不听指挥或者自作主张。

2. 案例分析

我们从生成的结果数据集中挑选了几个能展现打分特性的典型案例：

2.1. 完美达成案例 (Task_id: 9e3111b759961bbd)

- 用户请求：在仙林校区找评分 4.5 以上川菜馆，点水煮鱼和宫保鸡丁送仙林校区 2 栋，用微信支付。
- 分数表现：Tool: 1.0, Planning: 1.0, Goal: 1.0
- 评估情况：这是一个表现极佳的基准流水线。根据返回的 reasoning，Agent 精准执行了搜店、查菜、下单、支付四个工具步骤。由于行为序列与 expected_steps 像素级一致，所有各项均为满分，验证了系统对顺滑数据流的充分接纳。

2.2. 漏步导致部分完成的案例 (Task_id: 42966ceff3ff344b)

- 用户请求：帮我点一份张记烤肉店的双人套餐，送到仙林校区 1 栋，用微信支付。
- 分数表现：Tool: 0.0, Planning: 0.5, Goal: 0.75
- LLM-as-a-Judge 裁判解释：“Agent 成功查找了店铺和产品并下单，获得了编号，但完全遗漏了支付步骤，导致无法完成用户请求。”
- 案例分析：在没有采用 Ragas 粗暴 0 或 1 给分的前提下，我们的连续区间设计成功识别并反馈了“下单但未支付”具备的部分经济价值，为 Goal 分数赋予了 0.75，这比 0 要合理得多。

2.3. 幻觉偏离与逻辑断层案例 (Task_id: d6d4e1687347865b)

- **用户请求:** 帮我查一下订单 ORD-77102 送到了没有。
- **分数表现:** Tool: 0.0, Planning: 0.2, Goal: 1.0
- **LLM-as-a-Judge 裁判解释:** “Agent 没有查询明确要求的订单号 ORD-77102, 擅自更改订单号为 ORD-77120 进行查询。虽然最后答案已送达碰巧正确, 但规划严重偏离了用户意图。”
- **案例分析:** 这是一次经典的大模型幻觉。由于目标信息获取成功了, 所以 Goal 保住了满分。但底层的 Planning 分数由于逻辑的滑坡急速扣减到了 0.2。这说明了拆解独立评判 Planning 理性对于深挖潜藏的劣质 Agent 具有重大的意义。